C++ 语言程序设计笔记

语言与程序 ⟶ 类型与数据 ⟶ 指针与类 ⟶ 控制与函数 ⟶ 程序设计

第一章　基础知识

一、程序设计语言

程序设计语言分为机器语言、汇编语言和高级语言。

1. 源程序：扩展名为 `.cpp`；
2. 目标程序：扩展名为 `.obj`；
3. 可执行程序：扩展名为 `.exe`；
4. 翻译程序：包括汇编程序、编译程序和解释程序，C++ 属于编译型语言；
5. 链接程序：一般由 `build` 命令调用。

编辑源程序 → 源程序 → 翻译 → 目标程序 → 链接 → 可执行程序

二、程序组成

1. 注释：`//` 表示单行注释，`/* */` 表示跨行注释；
2. 编译预处理：以 `#` 开头，`include` 后接头文件；
3. 程序主体：`using namespace std;` 属于程序主体。

程序基本结构

```cpp
#include <iostream>

using namespace std;

int main() {
    return 0;
}
```

编译顺序：预编译器处理所有编译预处理指令 → 找到 `main()` 函数 → 编译。

三、内存

$$1\ \text{Byte} = 8\ \text{bit}, \qquad 1\ \text{KB} = 1024\ \text{B}, \qquad 1\ \text{MB} = 1024\ \text{KB}.$$

分区	内容与特点
代码区	程序指令
栈区（堆栈区）	局部变量、自动变量、局部常量；先入后出
全局变量区	全局变量（`extern`）
静态变量区	静态变量（`static`）、静态非基本常量
堆区	程序用 `new` 申请的空间，申请后必须用 `delete` 释放
常量区	字符串常量，以及全局常量、静态基本常量

寄存器型变量 `register` 存放于 CPU 寄存器；数量过多时，编译器会自动改按自动变量处理。全局变量与静态变量具有静态生存期并初始化为 0；局部变量具有动态生存期，建议显式初始化。

四、程序设计思想

维度	面向过程	面向对象
基本结构	顺序、选择、循环	类与对象；数据成员与成员函数
特性	模块化；自顶而下；逐步求精	封装、继承、多态
不足或意义	数据与过程分离，代码重用不足	模块化、软件复用
主要步骤	程序分解	OOA → OOD → OOP

程序设计应关注可靠性、成本效益、可理解性和可维护性。

五、词法标记

1. 字符集：大小写英文字母、数字字符和特殊字符；
2. 关键字：C++ 保留字；
3. 标识符：不能是关键字；首字符只能是字母或下划线；其余字符可为字母、下划线或数字；大小写敏感；不得含空格或特殊字符；
4. 分隔符：`( ) [ ] { } < > , : ; ' " // /* */`；
5. 空白：制表符、空格和空行。

六、运算符与表达式

运算符优先级

从高到低可记为：一元 → 算术 → 移位 → 关系 → 位 → 逻辑 → 三元 → 赋值 → 逗号。
一元、三元和赋值运算符通常从右向左结合，其他运算符通常从左向右结合。按“最大运算符”从左向右划分，例如 `a+++b` 等价于 `(a++) + b`。

- 赋值：区分 `=` 与 `==`；赋值运算符包括一般赋值和复合赋值；
- 自增：区分 `i++` 与 `++i`；
- 除法：两侧均为整型时作整数除法，否则作一般除法；注意验零；
- 取余：`%` 只能作用于整型，余数符号由被除数决定；
- 关系：优先级为 `> < >= <=` 高于 `== !=`；
- 逻辑：`!` 高于 `&&`，`&&` 高于 `||`；存在短路求值；

- 条件运算符：?:；
- 逗号运算符：无赋值或自增等副作用的子表达式不改变变量值；
- 溢出：按位数截断，可理解为对 $2^{\text{位数}}$ 取余。

整数除法与取余

```
int a = -4, b = 3;
std::cout << a / b << std::endl;          // -1
std::cout << 2 * b / a << std::endl;      // -1
std::cout << (float)a / b << std::endl;   // -1.333333
std::cout << a % b << std::endl;          // -1
std::cout << b % a << std::endl;          // 3
```

表达式由运算符、操作数和分隔符组成；语句通常为表达式加分号；块是由大括号括起的多条语句。

七、类型转换

- 隐式转换：二元运算符两侧类型按类型提升统一；逻辑型中 0 为 false，非 0 为 true；赋值时右值转为左值类型；
- C 风格：（类型名）表达式；
- C++ 风格：static_cast< 类型名 >(表达式)。

强制类型转换

```
int i = 97;
std::cout << (char)i << std::endl;
std::cout << static_cast<char>(i) << std::endl;

float f = 3.15;
std::cout << (int)f << std::endl;
std::cout << static_cast<int>(f) << std::endl;
```

八、编译预处理

头文件与宏

```
#include <iostream>       // 系统头文件
#include "my_header.h"    // 用户头文件

#define M 25
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#undef M
```

条件编译

```
#ifdef DEBUG
// 调试信息
#endif

#ifndef MY_HEADER_H
#define MY_HEADER_H
// 头文件内容
#endif
```

第二章　变量与常量

一、基本数据类型

基本类型包括整型 int、字符型 char、实型 float/double 和逻辑型 bool。

类型	常见长度（字节）	常见取值范围
char	1	$-2^7 \sim 2^7 - 1$
short int	2	$-2^{15} \sim 2^{15} - 1$
(long) int	4	$-2^{31} \sim 2^{31} - 1$
float	4	约 $-3.4 \times 10^{38} \sim 3.4 \times 10^{38}$
(long) double	8	约 $-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$

- short 只修饰 int；long 可修饰 int 和 double；
- unsigned 的取值从 0 开始；long long int 常占 8 字节；
- 实际长度与范围以编译环境为准，可用 sizeof 检查。

二、声明与初始化

变量

```
int value;
int a = 1;
int b(2);

typedef unsigned short int UINT16;
UINT16 index = 0;

std::cout << sizeof(int) << std::endl;
```

三、常量

符号常量必须在声明时初始化：

```
const int MAX_SIZE = 100;
```

1. 整型常数：十进制；二进制；八进制以 0 开头；十六进制以 0x 或 0X 开头；后缀可用 L、U；
2. 实型常数：小数形式或指数形式；默认 double，后缀可用 L、F；
3. 字符常数：'\a' 响铃、'\n' 换行、'\t' 制表、'\ooo' 八进制、'\xhh' 十六进制。

第三章　自定义数据类型

一、字符串

1. C 风格字符串

操作	字符数组	字符指针
初始化	`char sArray[] = "This is a book";`	`char* sPt = "This is a book";`
直接输出	`cout << sArray`	`cout << sPt`
直接输入	`cin >> sArray`	不可直接输入
直接修改	不可整体改写	`sPt = "OK"`
赋值	不可整体赋值	`sPt = sArray`

功能	函数原型	返回值	注意
长度	`int strlen(const char* s)`	长度	'\0' 不计入
复制	`char* strcpy(char* s1, const char* s2)`	s1	s1 空间足够
定长复制	`char* strncpy(char* s1, const char* s2, int n)`	s1	s1 空间足够
比较	`int strcmp(const char* s1, const char* s2)`	字符差值	从开头比较
连接	`char* strcat(char* s1, const char* s2)`	s1	s1 空间足够

2. C++ 风格字符串

std::string

```cpp
std::string str1;
std::string str2("Hello!");
std::string str3 = str2;

std::cin >> str1;
std::cout << str1;
std::getline(std::cin, str1);

str1 = "Hello!";
char ch1 = str1[1];
char ch2 = str1.at(1);
std::size_t n = str1.size();
```

字符串用关系运算符按字符的 ASCII 码依次比较；前部完全相同时，较长者大。

字符串与数字

```cpp
std::string s = "123";
int number = std::stoi(s);
std::string text = std::to_string(123);
```

二、数组

声明形式：类型标识符数组名 [常量表达式]。数组名不可被赋值，访问时必须检查越界。

数组初始化

```cpp
const int NUM = 5;
int scores[NUM] = {70, 100};
int zeros[NUM] = {0};
```

字符数组

```cpp
const int LEN_STR = 13;
char array1[LEN_STR] = {"Hello World!"};
char array2[LEN_STR] = "Hello World!";
char array3[LEN_STR] = {
    'H', 'e', 'l', 'l', 'o', ' ',
    'W', 'o', 'r', 'l', 'd', '!', '\0'
};
```

字符数组结尾必须有 '\0'，并为其留出空间。多维数组仅第一维可省略大小，其余维度必须给出。

三、枚举

枚举

```
enum weekday {sun, mon, tues, wed, thurs, fri, sat};
weekday day = sun;

enum weekday2 {sun2 = 7, mon2 = 1, tues2, wed2,
               thurs2, fri2, sat2};
day = (weekday)3;
int number = sun;
```

枚举元素本身是常量，默认从 0 开始递增；可指定起始值；可进行关系运算；枚举值赋给枚举变量时需要强制类型转换；通常按 int 存储。

四、结构与联合

结构

```
struct student {
    long num;
    char name[20];
};

student s1{}, s2{};
s2 = s1;
```

结构成员用圆点运算符 . 访问；结构指针用箭头运算符 -> 访问。只有结构类型相同的变量才能直接相互赋值。

联合

```
union uarea {
    char c_data;
    short s_data;
    long l_data;
};
```

联合各成员共享存储空间，其大小满足最大成员的存储与对齐要求。联合可无名称，作为无名联合使用。

第四章　指针与引用

一、指针

声明与初始化

```
int *pa1, *pa2;
char *pch1, *pch2;

int *pa = 0;
double *pb = NULL;
char *pc = nullptr;
```

指针保存地址；指向对象的解释方式由指针类型决定。未确定指向时应初始化为空指针，C++ 中推荐 `nullptr`。

- 一元运算：`*` 解引用、`&` 取地址、`++` 和 `--` 移动指针；
- `++`/`--` 优先级高于 `*` 和 `&`，故 `*p++` 等价于 `*(p++)`；
- 指针加减按所指类型的大小移动；
- 两指针不能相加，但同一数组中的两指针可相减，结果表示相隔的元素个数。

二、动态内存

语言	申请	释放
C	`malloc(sizeof(类型))`	`free(ptr)`
C++	`new 类型 (初值); new 类型 [大小]`	`delete ptr; delete[] ptr`

动态内存

```
int* pn = new int(10);
int* parr = new int[10];

delete pn;
delete[] parr;
```

三、引用

引用

```
int someInt = 0;
int &refInt = someInt;
```

引用必须在声明时初始化；引用关系定义后不可改指向其他对象。

四、多级指针

取地址运算符 `&` 与解引用运算符 `*` 可视为互逆操作，例如 `&*pa` 等价于 `pa`。

数组指针分析

可借用“微分/积分”记法辅助判断层级：

- 一般变量视为原函数，一级指针视为一阶微分，二级指针视为二阶微分；
- & 视为微分算子，* 视为积分算子；
- 定义数组或指针时，[] 提升一层；使用时，[] 降低一层；
- n 维数组可按 n 级指针理解；指针移动的单位由解引用后的对象类型决定。

该记法主要用于数组指针和多级指针读题。

多级指针例

```
int m[2][3] = {7};
int* pm[] = {
    m[1] - 1,
    &m[1][1],
    &m[0][0] + sizeof(m[1]) / sizeof(int),
    *m + 1,
    *(m + 1)
};

for (int i = 0; sizeof(int*) * i < sizeof(pm); i++) {
    *pm[i] += i + 1;
}
```

五、指针、引用与函数

指针和引用均可作为函数形参，通过地址传递修改实参。指针是 C 风格写法，引用是 C++ 风格写法；编写 C++ 时通常推荐引用。

六、const 与指针

名称	声明	含义
常指针	`const int *p1`	所指对象不可经该指针修改，指针可改指向
指针常量	`int *const p2`	指针不可改指向，所指对象可修改
常指针常量	`const int *const p3`	所指对象与指向均不可修改
常引用	`const int &ref`	不可经该引用修改对象

七、指针与结构体

结构体指针

```
struct student {
    int number;
};

```

```
student stu{};
student* ps = &stu;
student* dynamicStudent = new student;

ps->number = 1;
delete dynamicStudent;
```

链表的基本结构是在结构体中保存指向同类节点的指针。

第五章　类与继承

一、类

类是一组对象的抽象模型。对象的状态对应数据成员，行为对应成员函数。类的成员是抽象描述，对象的成员是具体实例。

类的声明

```
class Student {
public:
    void func();
protected:
    int protectedData;
private:
    int privateData;
};

void Student::func() {
}
```

成员函数在类内给出函数体时为内联函数；也可在类内声明、类外用作用域运算符 :: 定义。

二、构造函数

构造函数

```
class Clock {
public:
    Clock(int hour);
    Clock(int hour, int minute = 0);
    Clock(int hour, int minute, int second);
};

Clock::Clock(int hour) {
}
```

构造函数与类同名，无返回值，通常为公有成员。构造函数可按形参不同重载。若程序未定义构造函数，

编译器会补充默认构造函数。

构造函数歧义

无参构造函数与“所有参数都有默认值”的构造函数不能同时存在。调用无参构造时二者均可匹配，会产生歧义。

三、复制构造函数

复制构造函数

```
class Clock {
public:
    Clock(Clock& other);
};

Clock::Clock(Clock& other) {
}
```

未显式定义时，编译器会生成复制构造函数。若对象内部管理堆空间，则应显式定义复制构造函数，避免多个对象错误地共享同一堆地址。复制构造函数一般不重载。

四、析构函数

析构函数

```
class Clock {
public:
    ~Clock();
};
```

析构函数以~开头，无返回值，不能重载。

五、继承

继承可分为单继承和多继承。基类也称父类或超类，派生类也称子类。

继承

```
class TEllipse : public TShape {
};
```

派生类继承基类成员，但不继承构造函数、析构函数和默认赋值运算符。

访问属性	外部对象	派生类成员函数	本类成员函数
public	可	可	可
protected	不可	可	可
private	不可	不可	可

继承方式	基类 public	基类 protected	基类 private
public 继承	public	protected	不可访问
protected 继承	protected	protected	不可访问
private 继承	private	private	不可访问

区分访问属性与继承方式

先判断题目给的是成员的访问属性，还是类声明中的继承方式。访问属性决定“谁能访问”，继承方式决定基类成员进入派生类后的属性。

派生类中的同名成员会覆盖基类同名成员。

六、派生类构造与析构

1. 构造顺序：先基类，后派生类；
2. 若基类没有无参构造函数，派生类必须在初始化列表中显式调用基类构造函数；
3. 基类构造函数通常为 public，也可为 protected；
4. 析构顺序：先派生类，后基类。

派生类构造函数

```
class TEllipse : public TShape {
public:
    TEllipse(int longR, int shortR, int x, int y);
    ~TEllipse();
};

TEllipse::TEllipse(int longR, int shortR, int x, int y)
    : TShape(x, y) {
}
```

七、调用场景

函数	常见调用场景
构造函数	创建对象；动态分配；隐式转换
复制构造函数	初始化新对象；函数按值传递；返回对象
析构函数	形参销毁；临时变量销毁；delete；对象离开作用域

构造、复制与析构

```
class Simple {
public:
    int ID;

    Simple(int n) {
        std::cout << "Constructing Simple " << n << std::endl;
```

```
        ID = n;
    }

    Simple(Simple &s) {
        ID = s.ID;
        std::cout << "Copy Constructing Simple " << s.ID << std::endl;
    }

    ~Simple() {
        std::cout << "Destructing Simple " << ID << std::endl;
    }

    int getID() {
        return ID;
    }
};

void fun1(Simple s) {
    s.ID = 3;
}

Simple fun2(const int n) {
    Simple simple4(n);
    simple4.ID = 5;
    return simple4;
}

int main() {
    Simple simple1(1);
    Simple simple2 = simple1;
    simple2.ID = 2;
    fun1(simple2);
    fun2(4);

    Simple *pSimple = new Simple(6);
    std::cout << pSimple->getID() << std::endl;
    delete pSimple;
    return 0;
}
```

原笔记对该程序的计数为：构造函数 3 次、复制构造函数 3 次、析构函数 6 次；实际次数可能受编译器的返回值优化影响。

第六章　控制语句

控制结构分为顺序、选择和循环。算法可用伪代码或流程图表示；流程图中圆角矩形表示开始与结束，矩形表示执行，菱形表示判断，圆形表示连接。

一、选择结构

if–else

```
if (condition1) {
    // block 1
} else if (condition2) {
    // block 2
} else {
    // block 3
}
```

else 与最近的、尚未配对的 if 配对。

switch

```
switch (expression) {
case constant1:
    // block 1
    break;
case constant2:
    // block 2
    break;
default:
    // default block
}
```

- switch 表达式只能是字符型、整型或枚举型；
- case 后只能接字符型、整型或枚举型常量；
- 匹配后持续执行，直到遇到 break 或语句结束；
- 多个 case 执行同一段代码时，可连续写标签，只在末尾写一次 break；
- 所有 case 均不匹配时执行 default，没有 default 则跳过。

二、循环结构

三种循环

```
while (condition) {
    // loop body
}

do {
```

```
    // loop body
} while (condition);

for (initialization; condition; update) {
    // loop body
}
```

for 的执行顺序为：初始化 → 条件 → 循环体 → 更新 → 条件 → ···。三个表达式均可省略，但两个分号不可省略。

- 省略初始化：需在循环外赋初值；
- 省略条件：形成无限循环，除非通过 break 等跳出；
- 省略更新：可在循环体内修改循环变量；
- 省略初始化和更新：形式上接近 while。

break 跳出循环或结束 switch；continue 跳过本轮剩余语句，进入下一轮。

第七章　函数

函数由接口与函数体组成。接口包括函数名、返回值类型和形式参数表；形式参数表由参数类型、个数和顺序决定。

一、声明、定义与调用

函数

```
int add(int a, int b);      // 声明

int add(int a, int b) {     // 定义
    return a + b;
}
```

函数调用可作为函数语句、函数表达式或另一个函数的实参。实现调用的函数称主调函数，被调用的函数称被调函数。

二、作用域与可见性

1. 全局作用域，也称文件作用域；
2. 块作用域，从标识符定义处到本块结束；
3. 函数作用域；
4. 类作用域。

内层标识符与外层同名时，内层屏蔽外层。可用全局作用域运算符 :: 访问全局变量。

全局作用域运算符

```cpp
#include <iostream>

double pi = 3.14;

int main() {
    double pi = 3.1415926;
    std::cout << pi << std::endl;
    std::cout << ::pi << std::endl;
    return 0;
}
```

三、内联、递归与重载

- 内联函数：用 `inline` 定义，应先定义再使用；原笔记要求其中不含循环或 `switch`；
- 递归函数：直接或间接调用自身；必须具有终止条件；
- 函数重载：同名函数的参数个数、类型或顺序不同；不能只靠返回值类型区分。

第八章　编程技巧

一、输入输出格式

1. C 风格

C 风格输入输出

```cpp
printf("number: %04d, OK", 33);
putchar('c');

scanf("%d %c %f", &a, &b, &c);
char ch = getchar();
```

`printf` 与 `scanf` 的首个参数为格式字符串。多个输入占位符可用空格或逗号分隔。

2. C++ 风格

函数	作用	注意
`std::setw(int n)`	设置域宽	只影响后面一项
`std::setiosflags(std::ios::left)`	左对齐	持续生效
`std::setiosflags(std::ios::right)`	右对齐	持续生效
`std::setfill(char ch)`	设置填充字符	持续生效
`std::setprecision(int n)`	设置浮点有效数字	与 `fixed` 配合时控制小数位数
`std::setiosflags(std::ios::fixed)`	固定小数形式	配合 `setprecision`

读取带空格的一整行：

```cpp
std::getline(std::cin, str);
```

二、磁盘文件

文件可作为输入文件或输出文件。键盘是输入，显示器是输出，磁盘文件可兼具二者。文件分为文本文件和二进制文件。

文件流

```cpp
#include <fstream>

std::ifstream input("input.txt");
std::ofstream output("output.txt");
```

`ifstream` 提供读操作，`ofstream` 提供写操作。

三、随机数

随机数

```cpp
#include <cstdlib>
#include <ctime>

int main() {
    std::srand((unsigned)std::time(NULL));
    int randomNumber = std::rand() % 6;
    return 0;
}
```

对 n 取余可得到 $0 \sim n-1$ 范围内的数。

四、排序

标准库排序

```cpp
#include <algorithm>
#include <functional>
#include <vector>

std::vector<int> arr;
std::sort(arr.begin(), arr.end());
std::sort(arr.begin(), arr.end(), std::greater<int>());
```

快速排序

```cpp
void quickSort(int begin, int end) {
    if (begin >= end) return;
```

```

    int i = begin, j = end;
    int x = arr.at(begin);

    while (i < j) {
        while (i < j && arr.at(j) >= x) j--;
        if (i < j) arr.at(i++) = arr.at(j);

        while (i < j && arr.at(i) < x) i++;
        if (i < j) arr.at(j--) = arr.at(i);
    }

    arr.at(i) = x;
    quickSort(begin, i - 1);
    quickSort(i + 1, end);
}
```

冒泡排序

```
const int arraySize = 100;

void bubbleSort() {
    int pos = arraySize - 1;
    while (pos) {
        int bound = pos;
        pos = 0;
        for (int i = 0; i < bound; i++) {
            if (a[i] > a[i + 1]) {
                int hold = a[i];
                a[i] = a[i + 1];
                a[i + 1] = hold;
                pos = i;
            }
        }
    }
}
```

五、检查清单

程序检查

1. 检查边界条件：0、1、最大值、最小值、负数；
2. 和变量初始化为 0；
3. 除法前检查除数是否为 0，需要小数时使用浮点类型；
4. switch 中检查 break；

5. 浮点比较可先按需要缩放为整数，减少误差影响；
6. 检查输出格式和末尾是否多空格；
7. 没有思路时先列数学模型、变量关系和伪代码；
8. 读取整行（含空格）时使用 `getline` 或 `cin.getline`；
9. 大小写字母 ASCII 码通常相差 32，也可用 `tolower`/`toupper`。

段错误检查

常见原因：访问空指针、访问已释放内存、数组越界、修改字符串常量。

1. 检查所有指针是否初始化；
2. 检查数组访问是否越界；
3. 检查是否尝试修改字符串字面量，尤其是字符指针；
4. 检查递归是否有终止条件；
5. 检查内存是否重复释放。